第七章作业 参考答案

目录

1 图的存储结构与周游 **1**
(1) 相邻矩阵与度 1
(2) 邻接表 2
(3) DFS 与 BFS 序列 2
(4) 回路的判定 2
(5) 存储空间比较 3

2 项目依赖、拓扑排序与关键路径 **3**
(1) 顶点集与边集 3
(2) 拓扑排序伪代码与拓扑序列 3
(3) 最短总时间与关键路径 4

3 最小生成树与次小生成树 **4**
(1) Kruskal 算法 5
(2) Prim 算法 5
(3) 次小生成树 6

4 单源最短路径 **6**
(1) Dijkstra 过程 7
(2) 伪代码 7
(3) 负权边的反例 8

以下仅给出一种规范解法。等价的拓扑序、不同的并查集实现与最短路径松弛顺序均可判为正确。

1 图的存储结构与周游

第 (4) 问 “用 DFS 判断有向图是否有回路” 对应编程题：力扣 207. 课程表。

M207. 课程表

https://leetcode.cn/problems/course-schedule/

(1) 相邻矩阵与度

行为起点、列为终点，1 表示存在边 <行，列>：

```
      v0  v1  v2  v3  v4  v5
v0 [  0   1   0   1   0   0 ]
v1 [  0   0   1   0   1   0 ]
v2 [  0   0   0   0   0   1 ]
v3 [  0   0   0   0   1   0 ]
v4 [  0   0   1   0   0   1 ]
v5 [  1   0   0   0   0   0 ]
```

出度是该行 1 的个数，入度是该列 1 的个数：

顶点	v0	v1	v2	v3	v4	v5
出度	2	2	1	1	2	1
入度	1	1	2	1	2	2

出度之和与入度之和都等于边数 9。

(2) 邻接表

```
v0 -> 1 -> 3 -> ^
v1 -> 2 -> 4 -> ^
v2 -> 5 -> ^
v3 -> 4 -> ^
v4 -> 2 -> 5 -> ^
v5 -> 0 -> ^
```

(3) DFS 与 BFS 序列

DFS：访问 v0，先走最小邻接点 v1，再 v2，再 v5；v5 的邻接点 v0 已访问，回退到 v2、v1，访问 v1 的下一个邻接点 v4；v4 的邻接点 v2、v5 均已访问，回退到 v0，访问 v3；v3 的邻接点 v4 已访问，结束。

```
DFS: v0  v1  v2  v5  v4  v3
```

BFS：v0 出队，v1、v3 入队；v1 出队，v2、v4 入队；v3 出队，v4 已在队中；v2 出队，v5 入队；之后 v4、v5 依次出队。

```
BFS: v0  v1  v3  v2  v4  v5
```

(4) 回路的判定

DFS 访问到 v5 时，递归路径为 v0 -> v1 -> v2 -> v5，这四个顶点都” 已访问但尚未退出递归”。此时考察边 <5，0>，其终点 v0 正在递归栈中，因此 <5，0> 就是题目所说的边（回退边）。

理由：v0 仍在递归栈中说明存在从 v0 到 v5 的路径（即当前递归路径 v0 -> v1 -> v2 -> v5），再加上边 <5，0>，就构成回路 v0 -> v1 -> v2 -> v5 -> v0。反之，若 DFS 全程没有遇到这样的边，则图中无回路。对比第 (3) 问中其他” 指向已访问顶点” 的边：<4，2>、<4，5>、<3，4> 指向的都是已经退出递归的顶点，它们不构成回路。

(5) 存储空间比较

本图 `n = 6`、`e = 9`。

- 相邻矩阵：$n^2 \times 1 = 36$ 字节。
- 邻接表：顶点表 `n` 个表头指针共$4n = 24$ 字节；每个边结点含终点编号 2 字节和 `next` 指针 4 字节，共$6e = 54$ 字节；合计 78 字节。

本图用相邻矩阵更省。一般地，邻接表更省当且仅当

$$4n + 6e < n^2 \quad \Longleftrightarrow \quad e < \frac{n^2 - 4n}{6}.$$

即边数相对n^2 很少（稀疏图）时邻接表占优；本图$e = 9$ 远大于$(36 - 24)/6 = 2$，所以矩阵更省。若按教材习题的假设再为每条边存 2 字节权值，邻接表变为$4n + 8e$ 字节，矩阵变为$2n^2$ 字节，结论类似。

2 项目依赖、拓扑排序与关键路径

> 拓扑排序对应编程题：力扣 210. 课程表 II、OpenJudge 04084 拓扑排序。第 (3) 问对应力扣 2050. 并行课程 III。

M210. 课程表 II

https://leetcode.cn/problems/course-schedule-ii/

04084: 拓扑排序

http://cs101.openjudge.cn/practice/04084/

H2050. 并行课程 III

https://leetcode.cn/problems/parallel-courses-iii/

(1) 顶点集与边集

$$V = \{A, B, C, D, E, F, G, H\}$$

$$E = \{(A, D),\ (B, C),\ (B, D),\ (C, E),\ (D, E),\ (E, F),\ (F, G),\ (G, H)\}$$

(2) 拓扑排序伪代码与拓扑序列

基于入度的 Kahn 算法（教材 7.4.3 节的队列实现）：

```
TopoSort(G):
    for each vertex v in V:
        indeg[v] = inDegree(v)

    Q = queue of all vertices v with indeg[v] == 0
    L = empty list

    while Q is not empty:
        u = dequeue(Q)
        append u to L
        for each edge (u, v) in E:          // " 删除" u 及其出边
            indeg[v] = indeg[v] - 1
```

```
            if indeg[v] == 0:
                enqueue(Q, v)

    if |L| == |V|:
        return L
    else:
        report cycle                          // 仍有顶点未输出，图中有环
```

初始入度为 0 的顶点有 `A`、`B`。若同时出现多个入度为 0 的顶点按字母序取出，得到拓扑序列：

```
A  B  C  D  E  F  G  H
```

其他合法序列如 `B A C D E F G H`、`B C A D E F G H` 等同样正确。

(3) 最短总时间与关键路径

按拓扑序计算每项任务的最早开始时间 ES（所有前驱最早完成时间的最大值）和最早完成时间 EF = ES + 持续时间；再从后往前计算最迟完成时间 LF（所有后继最迟开始时间的最小值，无后继者取总时间）和最迟开始时间 LS = LF − 持续时间：

任务	持续时间	ES	EF	决定 ES 的前驱	LS	LF	松弛 LS − ES
A	3	0	3	无	0	3	0
B	2	0	2	无	0	2	0
C	4	2	6	B	2	6	0
D	3	3	6	A（B 也是前驱，但 EF 较小）	3	6	0
E	5	6	11	C、D	6	11	0
F	2	11	13	E	11	13	0
G	3	13	16	F	13	16	0
H	1	16	17	G	16	17	0

最短总时间为 EF 的最大值 **17**。

松弛为 0 的任务都在关键路径上，这里全部 8 项任务的松弛都是 0，但 `A` 与 `B` 对 `D` 的约束不同（`B -> D` 这条边上 `EF(B) = 2 < ES(D) = 3`，不是紧的）。沿“紧边”（前驱的 EF 等于后继的 ES）从源到汇追踪，得到两条关键路径：

```
A -> D -> E -> F -> G -> H      3 + 3 + 5 + 2 + 3 + 1 = 17
B -> C -> E -> F -> G -> H      2 + 4 + 5 + 2 + 3 + 1 = 17
```

任何一条关键路径上的任务延误，都会推迟整个项目。

3 最小生成树与次小生成树

> 最小生成树对应编程题：力扣 1584. 连接所有点的最小费用、OpenJudge 05442 兔子与星空。第 (3) 问的 “换边” 思路对应力扣 1489. 找到最小生成树里的关键边和伪关键边。

M1584. 连接所有点的最小费用

https://leetcode.cn/problems/min-cost-to-connect-all-points/

05442: 兔子与星空

http://cs101.openjudge.cn/practice/05442/

H1489. 找到最小生成树里的关键边和伪关键边

https://leetcode.cn/problems/find-critical-and-pseudo-critical-edges-in-minimum-spanning-tree/

(1) Kruskal 算法

按权值非降序扫描各边，用并查集判断两端是否已连通，连通则舍弃（会成环），否则选入：

扫描次序	边	权值	处理	选入后的连通分量
1	A-B	2	选入	{A,B}
2	C-E	2	选入	{A,B} {C,E}
3	B-C	3	选入	{A,B,C,E}
4	F-G	3	选入	{A,B,C,E} {F,G}
5	B-D	4	选入	{A,B,C,D,E} {F,G}
6	E-F	4	选入	{A,B,C,D,E,F,G}
—	A-C(5)、D-E(5)、C-D(6)、E-G(6)、D-F(7)、D-G(8)		两端已连通，舍弃	

已选入$n-1=6$ 条边，算法结束。最小生成树

$$T=\{A\text{-}B,\ C\text{-}E,\ B\text{-}C,\ F\text{-}G,\ B\text{-}D,\ E\text{-}F\},\qquad W(T)=2+2+3+3+4+4=18.$$

(2) Prim 算法

从 A 出发，每次在” 一端在树内、另一端在树外” 的边中取权值最小者：

步骤	树内顶点	候选边（权值）	加入的边
1	{A}	A-B(2)、A-C(5)	A-B
2	{A,B}	B-C(3)、B-D(4)、A-C(5)	B-C
3	{A,B,C}	C-E(2)、B-D(4)、C-D(6)	C-E
4	{A,B,C,E}	B-D(4)、E-F(4)、D-E(5)、C-D(6)、E-G(6)	B-D（与 E-F 同权，按表中次序）
5	{A,B,C,D,E}	E-F(4)、E-G(6)、D-F(7)、D-G(8)	E-F
6	{A,B,C,D,E,F}	F-G(3)、E-G(6)、D-G(8)	F-G

加入次序为 A-B, B-C, C-E, B-D, E-F, F-G，总权值 18。所选的 6 条边与 Kruskal 完全相同，只是加入次序不同。

一般地，当权值互不相同时最小生成树唯一（教材习题 11），两种算法必然得到同一棵树；本图有同权边，两种算法仍恰好得到同一棵树，但这不是必然的。

(3) 次小生成树

思路。任何一棵次小生成树都可以由最小生成树 `T` 换掉一条边得到：把一条非树边(u, v) 加入 `T` 会形成唯一的环，删去该环上（即 `T` 中 `u` 到 `v` 路径上）权值最大的树边 `f`，得到的仍是生成树，权值为$W(T) + w(u, v) - w(f)$。对所有非树边取其中最小的严格大于$W(T)$ 的候选即可。

非树边	权值	T 中两端点间路径	路径上最大树边	候选总权值
A-C	5	A-B-C	B-C (3)	20
D-E	5	D-B-C-E	B-D (4)	19
C-D	6	C-B-D	B-D (4)	20
E-G	6	E-F-G	E-F (4)	20
D-F	7	D-B-C-E-F	B-D 或 E-F (4)	21
D-G	8	D-B-C-E-F-G	B-D 或 E-F (4)	22

最小的严格增大的候选为 19。因此次小生成树可取

$$T' = \{A\text{-}B,\ C\text{-}E,\ B\text{-}C,\ F\text{-}G,\ D\text{-}E,\ E\text{-}F\}, \qquad W(T') = 19,$$

即用 `D-E`（5）替换 `B-D`（4）。

伪代码。

```
SecondMST(G):
    T = Kruskal(G)                          // 或 Prim
    W = totalWeight(T)
    best = INF

    for each edge e = (u, v) not in T:
        f = maxWeightEdgeOnPath(T, u, v)  // 在树 T 上 DFS/BFS 找 u 到 v 的路径，取最大边
        candidate = W + w(e) - w(f)
        if W < candidate < best:
            best = candidate
            bestTree = T - {f} + {e}

    return bestTree, best
```

每条非树边在树上找路径需$O(n)$，总时间$O(ne)$；用 LCA 倍增预处理路径最大边可降到$O(e \log n)$。

4 单源最短路径

对应编程题：力扣 743. 网络延迟时间、OpenJudge 05443 兔子与樱花。

M743. 网络延迟时间

https://leetcode.cn/problems/network-delay-time/

05443: 兔子与樱花

http://cs101.openjudge.cn/practice/05443/

(1) Dijkstra 过程

从 `A` 出发，每轮在未确定的顶点中取距离最小者加入集合 `S`，并用它的出边松弛。INF 表示当前尚不可达。

轮次	本轮确定顶点	S	d(A)	d(B)	d(C)	d(D)	d(E)	d(F)
初始	—	∅	0	INF	INF	INF	INF	INF
1	A	{A}	0	7	2	9	INF	INF
2	C	{A,C}	0	5	2	6	10	INF
3	B	{A,C,B}	0	5	2	6	10	INF
4	D	{A,C,B,D}	0	5	2	6	7	12
5	E	{A,C,B,D,E}	0	5	2	6	7	9
6	F	{A,C,B,D,E,F}	0	5	2	6	7	9

第 2 轮由 `C` 松弛：`d(B) = 2 + 3 = 5`、`d(D) = 2 + 4 = 6`、`d(E) = 2 + 8 = 10`。第 3 轮 `B` 的出边 `B-D`（5 + 2 = 7）、`B-E`（5 + 5 = 10）都不能改进。第 4 轮由 `D` 松弛：`d(E) = 6 + 1 = 7`、`d(F) = 6 + 6 = 12`。第 5 轮由 `E` 松弛：`d(F) = 7 + 2 = 9`。

最短路径长度与对应路径（由前驱数组回溯）：

终点	长度	一条最短路径
B	5	A-C-B
C	2	A-C
D	6	A-C-D
E	7	A-C-D-E
F	9	A-C-D-E-F

(2) 伪代码

```
Dijkstra(G, s):
    for each v in V:
        d[v] = INF
        parent[v] = NIL
    d[s] = 0
    S = empty set

    repeat |V| times:
        u = vertex not in S with minimum d[u]
        add u to S
        for each edge (u, v) leaving u, with v not in S:
            if d[u] + w(u, v) < d[v]:
                d[v] = d[u] + w(u, v)
                parent[v] = u

    return d, parent
```

用相邻矩阵并线性扫描选最小值，时间$O(n^2)$；用邻接表加最小堆，时间$O((n+e)\log n)$。

(3) 负权边的反例

取顶点 A、B、C，边 A-B 权 2、A-C 权 3、C-B 权 − 2。图中没有回路，自然没有负权回路。

从 A 出发运行上述算法：

轮次	确定顶点	d(A)	d(B)	d(C)
1	A	0	2	3
2	B	0	2	3
3	C	0	2（已在 S 中，不再更新）	3

算法输出 d(B) = 2，而真正的最短路径是 A-C-B，长度$3 + (-2) = 1$。

出错的步骤是第 2 轮：算法把当前距离最小的 B 加入 S，并认定 d(B) 已是最终值。这一贪心结论依赖"再经过任何其他顶点绕行都不会更短"，而这只在边权非负时成立。有了负权边 C-B，先经过距离更大的 C 再走负权边反而更短，B 被过早确定。即使实现中允许更新已在 S 中的顶点，从 B 出发的松弛也不会重新进行，依赖 B 的其他顶点仍会得到错误结果。

> 核对说明：第 2、3、4 题取自本次作业的原始题目，原答案经脚本逐题复核无误，此处补充了关键路径的最迟时间与松弛计算、Prim 算法过程（第 3 题第 (2) 问）、各顶点的最短路径及负权边反例（第 4 题第 (3) 问）。第 1 题为依据教材 7.3、7.4 节新增。